Project Log Analyzer

Name: Lam Bing Qing Andrew
Student: s5
Unit: CFC060524
Trainer: James

Table of Contents:

1. Introduction

2. Script
    2.1. Functions
        - oinput
        - tstampformat
        - printtstamp
        - usedcom
    2.2. Get user input
    2.3. Open and read auth.log
    2.4. For loop to print required logs

1. Introduction

This report aims to explain the accompanying script and what commands and functions were used to achieve its objectives. It also shows how python can be used to parse and return the required log entries from a log file like `auth.log`.

The script is configured to ask for user input, and depending on the input, either print out the entries of command usage from auth.log, or added, deleted users, password changes, `su`, `sudo` usage and failed attempts. All entries are read from `/var/log/auth.log`.

2. Script

2.1. Functions

- `oinput`

```
 8   # ~ Function to ask for user input. If the input is invalid, loop the function by calling it again.
 9   def oinput():
10
11       print('Options: ')
12       print('1. Print command usage only.')
13       print('2. Print parsed log.')
14       userin = input('Please choose an option (1 or 2): ')
15       print()
16
17       if userin != '1' and userin != '2':
18
19           print('Invalid input! Please provide a valid input! (1 or 2)')
20           print()
21           userin = oinput()
22
23       return userin
```

```
Options:
1. Print command usage only.
2. Print parsed log.
Please choose an option (1 or 2): 14

Invalid input! Please provide a valid input! (1 or 2)

Options:
1. Print command usage only.
2. Print parsed log.
Please choose an option (1 or 2): 2

Aug 14 15:02:06 User 'tc' used sudo on command: COMMAND=/usr/bin/ls
```

This function is used to ask for user input and save it to variable `userin`. If the input is invalid(not 1 or 2), the function informs the user and calls on itself to provide a loop until a valid input is detected. The variable is then returned to the main script.

- `tstampformat`

```
25  # ~ Function tstampformat is used to verify the format of the time stamp in auth.log.
26  # ~ Currently the function is only able to detect the complete time stamp if it has the time in
27  # ~ the format XX:XX:XX in its last field.
28  def tstampformat():
29
30      index = 0
31
32      for chk in word:
33
34          if re.search("[0-9]{2}:[0-9]{2}:[0-9]{2}", chk):
35
36              return index
37              break
38
39          else:
40
41              index += 1
```

The function here is used to detect the format of timestamp in the log. It can currently only detect the format if the timestamp has the time in the format `XX:XX:XX` in the last field.

- `printtstamp`

```
43  # ~ Using the information from tstampformat, this function then adds the time stamp to a string and returns it.
44  def printtstamp():
45
46      tstamp = ''
47
48      for tcount in range(tformat + 1):
49
50          tstamp += word[tcount] + ' '
51
52      tstamp = tstamp.strip()
53
54      return tstamp
```

`printtstamp` uses the information from the previous function and adds the timestamp into a string before returning it. Using the above two functions, the script is able to handle logs with different timestamp formats as seen below.

```
2024-07-01T08:23:21.704377-04:00 User 'kali' us
2024-07-01T08:24:43.110977-04:00 User 'kali' us
```

```
Aug 14 15:02:06 User 'tc' use
Aug 14 15:04:19 User 'tc' use
```

- `usedcom`

```
56  # ~ This function is used add the commands used to a string and return it to the main script.
57  def usedcom():
58
59      command = ''
60
61      for cword in word:
62
63          if 'COMMAND' in cword or 'COMMAND' in command:
64
65              command += cword + ' '
66
67      command = command.strip()
68
69      return command
```

This function has a similar function to `printtstamp`. However, instead of adding the timestamp, the function detects the command used and adds it to a string before returning it to the script

2.2. Get user input

```
71  # ~ Get user input
72  uinput = int(oinput())
```

The script begins by getting the user input through `oinput` and passing the result through the `int` inbuilt function to change the data type to an integer. It is then saved to the variable `uinput`.

2.3. Open and read auth.log

```
74  # ~ Opening auth.log from /var/log and reading it into a list variable.
75  with open('/var/log/auth.log', 'r') as file:
76
77      data = file.readlines()
```

Next the script opens `/var/log/auth.log` and reads its data, with each entry being an index on a list. The list is saved to variable `data`.

2.4. For loop to print required logs

```
80  for line in data:
81
82      cleanline = line.strip()
83      word = cleanline.split()
84      tformat = tstampformat()
85
86      # ~ Print sudo usage.
87      if 'sudo' in word[tformat + 2] and 'TTY' in word[tformat + 5]:
88
89          print(printtstamp(), 'User', '\'' + word[tformat + 3] + '\'', 'used sudo on command:', usedcom())
90
91      # ~ Print failed sudo attempts.
92      elif 'sudo' in word[tformat + 2] and 'NOT' in word[tformat + 6]:
93
94          print(printtstamp(),'ALERT! User', '\'' + word[tformat + 3] + '\'', 'failed to use sudo! The attempted command was:', usedcom())
95
96      # ~ Print parsed log.
97      if uinput == 2:
98
99          # ~ Print added users.
100         if 'useradd' in word[tformat + 2]:
101
102             userdata = word[tformat + 5].split('=')
103             username = userdata[1].split(',')
104             print(printtstamp(), 'New user', '\'' + username[0] + '\'', 'has been added.')
105
106         # ~ Print deleted users.
107         elif 'userdel' in word[tformat + 2] and 'delete' in word[tformat + 3]:
108
109             print(printtstamp(), 'User', word[tformat + 5], 'was deleted.')
110
111         # ~ Print password changes.
112         elif 'passwd' in word[tformat + 2] and 'changed' in word[tformat + 5]:
113
114             print(printtstamp(), 'Password was changed for user', '\'' + word[-1] + '\'.')
115
116         # ~ Print su usage.
117         elif 'su:' in word[tformat + 2] and 'su:session' in word[tformat + 3]:
118
119             user1 = word[-1].split('(')
120             user2 = word[-3].split('(')
121
122             print(printtstamp(), 'User', '\'' + user1[0] + '\'', 'used su to switch to user', '\'' + user2[0] + '\'.')
123
124         # ~ Print failed su attempts.
125         elif 'su:' in word[tformat + 2] and 'FAILED' in word[tformat + 3]:
126
127             user1 = word[tformat + 6].split(')')
128             print(printtstamp(), 'User', '\'' + word[-3] + '\'', 'failed su to user', '\'' + user1[0] + '\'.')
129
```

The script then loops for each entry in the list, cleaning the lines of any leading or trailing whitespaces and dividing them by words, to better sort them.

Depending on the user input from the start, the script either prints out only the `sudo` command usage and attempts, or added, deleted users, password changes, `su`, `sudo` usage and failed attempts.

```
Please choose an option (1 or 2): 1

Aug 14 15:02:06 User 'tc' used sudo on command: COMMAND=/usr/bin/ls
Aug 14 15:04:19 User 'tc' used sudo on command: COMMAND=/usr/sbin/adduser test
Aug 14 15:05:52 User 'tc' used sudo on command: COMMAND=/usr/sbin/deluser test
Aug 14 15:06:18 User 'tc' used sudo on command: COMMAND=/usr/sbin/adduser test
Aug 14 15:06:54 User 'tc' used sudo on command: COMMAND=/usr/sbin/deluser test
```

```
Please choose an option (1 or 2): 2

Aug 14 15:02:06 User 'tc' used sudo on command: COMMAND=/usr/bin/ls
Aug 14 15:03:11 User 'tc' failed su to user 'root'.
Aug 14 15:04:19 User 'tc' used sudo on command: COMMAND=/usr/sbin/adduser test
Aug 14 15:04:19 New user 'test' has been added.
Aug 14 15:04:44 Password was changed for user 'test'.
```